

KHULNA UNIVERSITY OF ENGINEERING & TECHNOLOGY
Department of Computer Science and Engineering

CSE 4102: Computer Graphics and Image Processing Laboratory
4th Year 1st Term

SENTINEL

A Fully Procedural Game Engine and Open-World Military Shooter

Submitted by

Md. Shifat Hasan

Roll: 2107067

Submitted to

Md Tajmilur Rahman

Md. Mubtashim Abrar Nihal

Instructors, Department of Computer Science and Engineering



Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna 9203, Bangladesh

October 7, 2026

Abstract

Most video games ship with gigabytes of files made by artists: 3D models, photographs for surfaces, recorded sounds. *Sentinel* takes the opposite route. It is a complete game engine and an open-world military shooter in which *nothing* is loaded from a model, image or sound file. Every tree, building, soldier, vehicle, surface, cloud and sound effect is created while the program starts, from short mathematical recipes and a random seed. Only the small text files holding the shader programs are read from disk.

The engine is written in the Odin language on OpenGL 4.1, the newest version macOS offers (it has no compute shaders), and is organised in six layers that may only depend on the layers beneath them. A deferred renderer records what every pixel is made of and then lights it with a sun and moon, spot and point lamps, cascaded shadow maps, ambient occlusion, screen-space reflections, an atmospheric sky with wind-driven clouds, fog, bloom, tone mapping, colour grading and anti-aliasing. Surfaces come from 18 texture recipes built on periodic gradient noise, domain warping and cellular noise; normal maps are derived from height fields with a Scharr filter. Objects and characters are tables of primitives, each placed by a chain of 4×4 transformation matrices (scale, rotate, move, hierarchy, view, projection), and lit by six selectable illumination models (Lambert, Phong, Blinn–Phong, Oren–Nayar, Cook–Torrance, subsurface) under directional, point, spot and area lights.

On top of the engine sit collision, a ladder state machine, four ground vehicles and a helicopter, enemy soldiers with stealth rules, synthesised audio, and a five-objective rescue mission in the spirit of *Project I.G.I.* The work was measured and verified, not only drawn: at 1080p the GPU needs 10.9 ms per frame, and 281 unit tests, 30 GL checks, 8 318 texture checks (including exact tiling of every material), 318 567 renderer checks and two scripted full playthroughs pass. The report explains every technique in plain language, with its governing equations, diagrams and screenshots, and documents a debugging case in which a star in the fog colour painted a bright vertical line across the night scene.

Keywords: procedural generation, game engine, OpenGL, deferred rendering, physically based shading, noise, texture synthesis, real-time graphics.

Contents

	PAGE
Abstract	i
Contents	ii
List of Tables and Figures	iii
CHAPTER I Introduction	1
1.1 Introduction	1
1.2 Background and Problem Statement	1
1.3 Objectives	1
1.4 Scope	2
1.5 Unfamiliarity of the Problem and its Solution	2
1.6 Project Planning and Work Distribution	2
1.7 Applications of the Work	2
1.8 Organization of the Project	2
CHAPTER II Related Works	3
2.1 Introduction	3
2.2 Related Works	3
2.3 Discussion	3
CHAPTER III Methodology	4
3.1 Introduction	4
3.2 Detailed Methodology	4
CHAPTER IV Implementation, Results and Discussions	15
4.1 Introduction	15
4.2 Experimental Setup	15
4.3 Evaluation Metrics	15
4.4 Dataset	16
4.5 Implementation and Results	16
4.6 Objective Achieved	17
CHAPTER V Societal, Health, Environment, Safety, Ethical, Legal and Cultural Issues	18
5.1 Intellectual Property Considerations	18
5.2 Ethical Considerations	18
5.3 Safety Considerations	18
5.4 Legal Considerations	19
5.5 Impact on Societal, Health and Cultural Issues	19
5.6 Impact on the Environment and Sustainability	19
CHAPTER VI Addressing Complex Engineering Problems and Activities	20
6.1 Complex Engineering Problems	20
6.2 Complex Engineering Activities	20
CHAPTER VII Conclusions	21
7.1 Summary	21
7.2 Limitations	21
7.3 Recommendations and Future Works	21
References	22

List of Tables

Table No.	Description	Page
1.1	Tools and technologies used	2
2.1	Comparison with related approaches (typical characteristics)	3
3.1	The six illumination models compared	12
4.1	Experimental setup	15
4.2	GPU time per pass, 1080p benchmark	16
6.1	Complex engineering problem attributes	20

List of Figures

Figure No.	Description	Page
1.1	Work plan by day (the single author performed every task).	2
3.1	The six layers (dependencies point downward) and the main loop.	4
3.2	Objects built from transformed primitives; the turret and gun inherit the hull's matrix.	6
3.3	How an object is created and combined: the chain of transformations from a primitive to a pixel.	7
3.4	Noise ingredients (illustration of the shader equations): gradient, fractal, warped, ridged, cellular.	9
3.5	The passes of one frame, left to right.	10
3.6	The geometry of lighting: surface normal, light, view and half vectors; sun, lamp, spotlight and rectangular panel.	10
3.7	The same painted-metal sphere under the six illumination models (left to right: Lambert, Phong, Blinn-Phong, Oren-Nayar, Cook-Torrance, Subsurface), lit by the same sun and sky. Lambert has no highlight; Phong and Blinn-Phong add a hot spot; Oren-Nayar flattens the falloff; Cook-Torrance adds Fresnel and a microfacet highlight.	12
3.8	Two curves behind the lighting: distance falloff and glancing-angle reflection.	13
4.1	GPU time per pass (total 10.91 ms of the 16.7 ms budget).	16
4.2	The engine and game in action; (h) and (i) show the streak bug and its fix.	17
6.1	The verification loop used for every feature.	20

CHAPTER I

Introduction

1.1 Introduction

Imagine a film studio that buys no props: a recipe book tells assistants how to build every tree and cloud, and a different “seed” gives a different but believable forest. *Sentinel* is that studio for a game: a *game engine* (the machinery that draws, simulates and sounds a world) plus a first-person military mission. *Procedural* means shapes, colours and sounds are calculated by code, not stored in files. (A **GPU** is a second processor with thousands of tiny cores; a **shader** is the small program each core runs per pixel.)

1.2 Background and Problem Statement

Graphics courses teach lighting, textures and noise one at a time; a product needs them working *together* 60 times a second, usually with art loaded from files. The problem: *can a complete, good-looking, playable 3D game be produced by code alone on a student laptop?* Limits: no pre-saved model, image or precomputed file, and OpenGL 4.1 without compute shaders.

1.3 Objectives

1. Build a layered engine with downward-only dependencies.
2. Generate all meshes, textures and sounds from code and seeds.
3. Implement a modern renderer (PBR, shadows, sky).
4. Build a playable game with vehicles, ladders, doors and a mission.
5. Verify it with tests and a 16.7 ms frame budget.

1.4 Scope

The scope is the whole engine and one game (Table 1.1); networking, multiplayer and hand-made art are out of scope.

Table 1.1: Tools and technologies used

Tool	Role (plain words)
Odin language	The project language: fast like C, readable like a script.
OpenGL 4.1 core	The standard way to talk to the graphics card on macOS.
GLFW, miniaudio	Window and input; sound output.

1.5 Unfamiliarity of the Problem and its Solution

The design follows published theory, not an existing code base: commercial engines import art, and demo-scene shooters are not readable. Here “no files” is enforced everywhere.

1.6 Project Planning and Work Distribution

One student did all the work; Fig. 1.1 is rebuilt from the 138 commits.

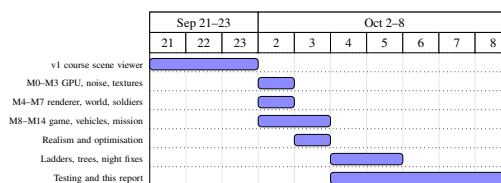


Figure 1.1: Work plan by day (the single author performed every task).

1.7 Applications of the Work

A teaching tool and a base for small games.

1.8 Organization of the Project

Chapters II to IV give related work, design and results; V and VI discuss social and engineering issues; VII concludes.

CHAPTER II

Related Works

2.1 Introduction

Sentinel combines noise, physically based shading, real-time pipelines and procedural games.

2.2 Related Works

Noise. Perlin’s noise [1] blends random gradients on a grid; summing scales gives natural detail. Worley’s cellular noise [2] gives pebbles and leaves; domain warping [13] bends patterns to hide the grid. Sentinel uses all three, made *periodic* so textures tile.

Shading. Cook–Torrance with GGX [3–5] is the standard reflection model; Oren–Nayar [8] suits rough cloth.

Pipeline and games. Deferred shading [14], cascaded shadows [7], Hillaire’s sky [6], ACES [9] and FXAA [11] form the pipeline; *.kkrieger* (2004) generated a shooter at run time and *Project I.G.I.* (2000) inspired the mission.

2.3 Discussion

Table 2.1 shows the gap.

Table 2.1: Comparison with related approaches (typical characteristics)

Feature	Tutorials	Commercial	.kkrieger	Sentinel
No imported art	Mixed	No	Yes	Yes
Complete game loop	No	Yes	Yes	Yes
Day/night, clouds, shadows	Rare	Yes	No	Yes
Works without compute shaders	Varies	Rarely	Yes	Yes
Written to be read	Yes	No	No	Yes

CHAPTER III

Methodology

3.1 Introduction

One frame is an assembly line: the game decides what exists, the engine builds it, the renderer paints it.

3.2 Detailed Methodology

3.2.1 Overall Framework

Figure 3.1: a lower layer never knows about a higher one. Source/Main.odin is only a table of contents: initialise, then repeat *input*, *update*, *render*, then shut down. Procedures stay near 40 lines and every GL call is checked.

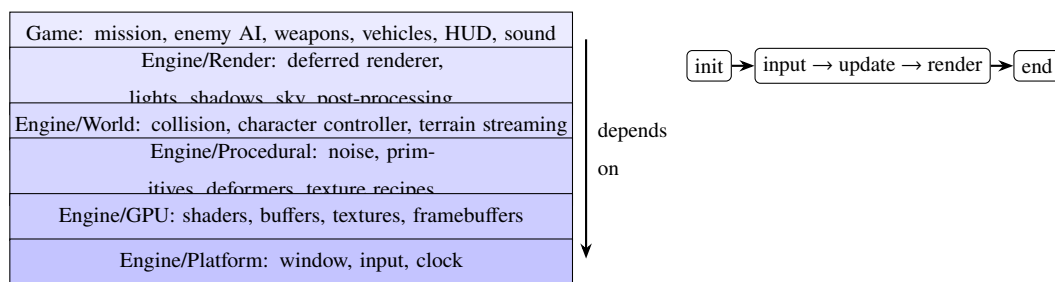


Figure 3.1: The six layers (dependencies point downward) and the main loop.

3.2.2 Transformations: Creating and Combining Objects

Every object is a *table* of parts. A part is a primitive (box, sphere, cylinder, cone, capsule, torus, wedge), optional deformers, a transform and a material. A point is written in *homogeneous*

coordinates $\mathbf{p} = (x, y, z, 1)^\top$ so that moving, turning and scaling are all multiplications by 4×4 matrices and can be chained into one matrix:

$$T(\mathbf{t}) = \begin{pmatrix} I & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{pmatrix}, \quad S(\mathbf{s}) = \text{diag}(s_x, s_y, s_z, 1), \quad R_y(\theta) = \begin{pmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \quad (3.1)$$

R_x and R_z are the same pattern turned about their own axes. The transform of one part, as built by the engine, is

$$M_{\text{part}} = T(\mathbf{t}) R_z(\gamma) R_y(\beta) R_x(\alpha) S(\mathbf{s}), \quad \mathbf{p}_{\text{object}} = M_{\text{part}} \mathbf{p}_{\text{part}}. \quad (3.2)$$

Matrices act on the point from the right, so the order of events is *right to left*: the part is first scaled (stretched), then turned about x , then y , then z , and finally moved. Reversing the order gives a different shape, because rotations and scaling do not commute.

In plain words

Think of a LEGO model: each brick is made at the origin, squeezed or stretched, turned, and only then carried to its place. Because the *last* step is the move, the turn happens around the brick's own centre, not around the world's centre.

Worked example (numbers checked by script)

A box vertex $(1, 0, 0)$ with $\mathbf{s} = (2, 1, 1)$, $\beta = 90^\circ$ and $\mathbf{t} = (5, 0, 0)$: scaling gives $(2, 0, 0)$; $R_y(90^\circ)$ maps $(x, z) \rightarrow (x \cos \beta + z \sin \beta, -x \sin \beta + z \cos \beta)$ giving $(0, 0, -2)$; the move gives $(5, 0, -2)$. (Checked by script.)

Normals need a different matrix. A surface normal must stay perpendicular to the surface. If the surface is stretched, applying M to the normal would tilt it. The engine therefore transforms normals with the *inverse transpose* of the upper 3×3 block L of M :

$$\hat{n}' = \frac{(L^{-1})^\top \hat{n}}{\|(L^{-1})^\top \hat{n}\|}. \quad (3.3)$$

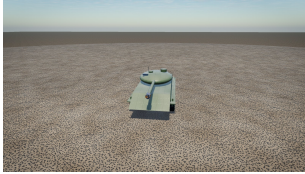
For $S = \text{diag}(2, 1, 1)$ and $\hat{n} = (1, 1, 0)/\sqrt{2}$: the correct normal is $(0.5, 1, 0)/1.118 = (0.447, 0.894, 0)$, whereas naively applying S gives $(0.894, 0.447, 0)$, which is no longer perpendicular to the stretched tangent $(2, -1, 0)$ (dot product $1.34 \neq 0$; for the correct normal it is 0). If $\det L < 0$ the object is mirrored; the engine then reverses every triangle's winding and the tangent's handedness so faces still point outward.

Combining parts. All parts of an object that share a material are appended into *one* mesh (each vertex multiplied by its part matrix), so a whole building costs one draw call per material, not one per brick. The finished object is placed in the world by one more matrix, $M_{\text{world}} = T(x, \text{ground}, z) R_y(\text{yaw})$, and *instancing* draws the same mesh at many placements, each with its own M_{world} (one call for thousands of trees).

Hierarchies. Moving parts hang below their parent, so their matrices are *products down the chain*. A tank’s gun is

$$M_{\text{gun}} = \underbrace{T(\mathbf{p})R_y(\psi)R_x(-\phi)R_z(\rho)}_{\text{hull}} \underbrace{T(\mathbf{c}_t)R_y(\theta_t)}_{\text{turret}} \underbrace{T(\mathbf{c}_g)R_x(-\theta_g)}_{\text{gun}}, \quad (3.4)$$

with hull position $\mathbf{p}$, yaw ψ , pitch ϕ , roll ρ , turret pivot $\mathbf{c}_t$ and yaw θ_t , and gun pivot $\mathbf{c}_g$ and elevation θ_g . Turning the hull therefore carries the turret and gun with it, while the turret turns alone about its own pivot. A soldier’s limb is built the same way from the posed joint positions: its matrix has $\mathbf{t}$ = the upper joint and its local y axis along (lower joint – upper joint), so each bone follows its joints.



(a) Tank: hull, turret and gun



(b) Truck: body, cab and wheels

Figure 3.2: Objects built from transformed primitives; the turret and gun inherit the hull’s matrix.

Deformers are one-line mappings applied in the part’s own space before M_{part} : *taper* scales x, z by a factor that varies linearly with height, *twist* rotates by an angle growing with height, *bend* wraps the axis onto a circle, and *noise displacement* pushes each vertex along its normal by at most A . In formulas, with height y in $[0, h]$:

$$\text{taper: } (x, z) \mapsto \left(1 + (k_t - 1)\frac{y}{h}\right)(x, z), \quad \text{twist: } \theta(y) = \Theta \frac{y}{h}, \quad \text{bend: } \kappa = \frac{1}{R}, \quad (3.5)$$

and the noise displacement is

$$\mathbf{p}' = \mathbf{p} + A \nu(f\mathbf{p}) \hat{n}, \quad \|\mathbf{p}' - \mathbf{p}\| \leq A. \quad (3.6)$$

Soldiers' legs reach their targets by two-bone inverse kinematics: for bone lengths a, b and hip-to-target distance c , the law of cosines gives the hip angle α between the target direction and the thigh,

$$\cos \alpha = \frac{a^2 + c^2 - b^2}{2ac}. \quad (3.7)$$

From the world to the screen. Drawing a vertex multiplies it by the chain model, view and projection. The camera's *view* matrix is a look-at transform (move the eye to the origin, turn the target onto $-z$), and the *perspective* matrix, built from the focal factor

$$f = \frac{1}{\tan(\text{fov}/2)}, \quad (3.8)$$

divides by depth:

$$\mathbf{p}_{\text{clip}} = P V M_{\text{world}} M_{\text{part}} \mathbf{p}, \quad P = \begin{pmatrix} f/a & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & \frac{n+r}{n-r} & \frac{2nr}{n-r} \\ 0 & 0 & -1 & 0 \end{pmatrix}, \quad \mathbf{p}_{\text{screen}} = \frac{(x,y)}{w}. \quad (3.9)$$

Here a is the aspect ratio, n, r the near and far planes, and w the clip-space fourth coordinate (equal to the distance in front of the camera), so far objects shrink. Binocular zoom by a factor k narrows the view,

$$\tan \frac{\text{fov}'}{2} = \frac{1}{k} \tan \frac{\text{fov}}{2}, \quad (3.10)$$

so a 72° view at $4\times$ becomes 20.6° .

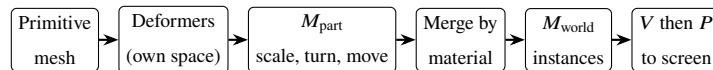


Figure 3.3: How an object is created and combined: the chain of transformations from a primitive to a pixel.

3.2.3 Procedural Textures

Each material is a *recipe*: a small shader that answers “what is the colour, bumpiness, shininess and crevice-darkness at this point?”. Each recipe is baked at start-up into a 1024×1024 picture (22 materials from 18 recipes: concrete, rock, sand, dirt, asphalt, metals, paints, wood, bark, leaves, needles, grass, canvas, camo, rubber, skin, glass) and keeps it on the GPU.

Gradient noise (a rubber sheet pinned at random slopes) blends the four corner gradients of a grid cell with the smooth fade curve

$$s(t) = 6t^5 - 15t^4 + 10t^3, \quad s(0) = 0, \quad s(1) = 1, \quad s'(0) = s'(1) = s''(0) = s''(1) = 0, \quad (3.11)$$

scaled by $\sqrt{2}$ so values lie in $[-1, 1]$. Fractal noise sums octaves, each at double frequency and half strength:

$$F(\mathbf{x}) = \frac{\sum_{i=0}^{n-1} 2^{-i} v(2^i \mathbf{x})}{\sum_{i=0}^{n-1} 2^{-i}}, \quad F \in [-1, 1]. \quad (3.12)$$

Choosing gradients from the cell index *modulo the period* P makes the picture tile with no seam (tested exactly):

$$F(\mathbf{x} + P\mathbf{e}) = F(\mathbf{x}) \quad \text{for every whole-number shift } \mathbf{e}. \quad (3.13)$$

The materials come from three variants (Fig. 3.4): *domain warping* swirls the pattern (stone, wood, soil), *ridged* noise makes cracks and bark, and *cellular* noise gives leaves and pebbles,

$$F_{\text{warp}}(\mathbf{x}) = F(\mathbf{x} + s[F_1(\mathbf{x}), F_2(\mathbf{x})]), \quad F_{\text{ridge}}(\mathbf{x}) = \frac{\sum_{i=0}^{n-1} 2^{-i} (1 - |v_i|)^2}{\sum_{i=0}^{n-1} 2^{-i}}. \quad (3.14)$$

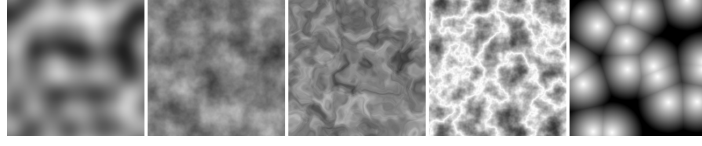


Figure 3.4: Noise ingredients (illustration of the shader equations): gradient, fractal, warped, ridged, cellular.

Bumps without geometry. A recipe’s height map becomes a *normal map* that fools the lighting into seeing bumps. The slope uses the Scharr 3×3 filter, the most direction-fair small filter, and crevices (height below the neighbours’ average) darken the albedo and ambient occlusion:

$$h_x \approx \frac{3(h_{NE} + h_{SE} - h_{NW} - h_{SW}) + 10(h_E - h_W)}{32 \Delta N}, \quad \hat{n} = \frac{(-k h_x, -k h_y, 1)}{\|\cdot\|}, \quad (3.15)$$

where Δ is one texel and N the cells per tile, so bump strength k is scale-independent. Triplanar mapping projects the picture along three axes and blends them by

$$w_i = \frac{|n_i|^4}{\sum_{j \in \{x, y, z\}} |n_j|^4}, \quad i \in \{x, y, z\}, \quad (3.16)$$

so deformed shapes are never stretched.

3.2.4 The Rendering Pipeline

Deferred shading photographs a “data poster” of the scene first (each pixel’s colour, direction and shininess) and lights it once, which makes many lights cheap. The poster is the *G-buffer*: albedo, octahedral-packed normal with roughness and metalness, emission with occlusion, and depth (from which position is rebuilt). Fig. 3.5 lists the passes. Distant things fade into haze:

$$C = T C_{\text{surface}} + (1 - T) C_{\text{haze}}, \quad T = e^{-k d}. \quad (3.17)$$

Finally the HDR image is compressed to screen range by the ACES curve [9], graded, dithered [10] and smoothed (FXAA [11]):

$$C_{\text{out}} = \text{clamp}\left(\frac{x(2.51x + 0.03)}{x(2.43x + 0.59) + 0.14}\right), \quad x = \text{exposure} \times \text{radiance}. \quad (3.18)$$

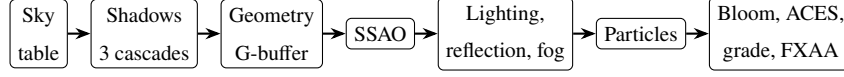


Figure 3.5: The passes of one frame, left to right.

3.2.5 Illumination Models and Light Sources

The lighting pass visits every pixel of the data poster and adds up the contribution of each light, then the sky's ambient light and the surface's own emission:

$$L_o = \sum_{j=1}^{N_L} f(\mathbf{l}_j, \mathbf{v}) L_j \text{att}_j \text{sh}_j \max(0, \mathbf{n} \cdot \mathbf{l}_j) + \underbrace{\text{ambient} \cdot \text{AO}}_{\text{sky}} + \mathbf{e}, \quad N_L = \text{number of lights}. \quad (3.19)$$

Here f is the *illumination model* (the BRDF, the share of light from direction $\mathbf{l}$ that leaves toward the eye $\mathbf{v}$), L_j the light's colour times intensity, att_j its distance falloff, $\text{sh}_j \in [0, 1]$ its shadow, $\mathbf{n} \cdot \mathbf{l}$ Lambert's cosine (light spreads over a larger area when it arrives at a slant), and $\mathbf{e}$ the emission of lamps and windows. Figure 3.6 shows the four vectors every model uses; $\mathbf{h} = (\mathbf{l} + \mathbf{v}) / \|\mathbf{l} + \mathbf{v}\|$ is the half vector.

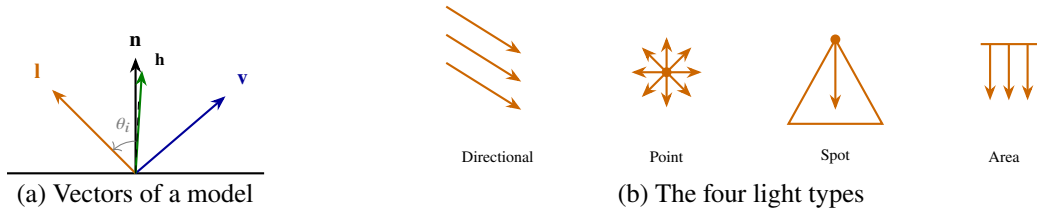


Figure 3.6: The geometry of lighting: surface normal, light, view and half vectors; sun, lamp, spotlight and rectangular panel.

The six illumination models are chosen per pixel from the G-buffer (Fig. 3.7). With $\mathbf{c}_d$ the diffuse colour (metals have none) and F_0 the base reflectance (Subsurface replaces the

lighting cosine, so it is written for that term):

$$f_{\text{Lambert}} = \frac{\mathbf{c}_d}{\pi} \quad (3.20)$$

$$f_{\text{Phong}} = \frac{\mathbf{c}_d (1 - F_0)}{\pi} + F_0 \frac{m + 2}{2\pi} (\mathbf{r} \cdot \mathbf{v})^m, \quad \mathbf{r} = 2(\mathbf{n} \cdot \mathbf{l}) \mathbf{n} - \mathbf{l} \quad (3.21)$$

$$f_{\text{Blinn-Phong}} = \frac{\mathbf{c}_d (1 - F_0)}{\pi} + F_0 \frac{m + 8}{8\pi} (\mathbf{n} \cdot \mathbf{h})^m \quad (3.22)$$

$$f_{\text{Oren-Nayar}} = \frac{\mathbf{c}_d}{\pi} \left(A + B \max(0, \cos \Delta\varphi) \sin \alpha \tan \beta \right) \quad (3.23)$$

$$f_{\text{Cook-Torrance}} = \frac{(1 - F_l)(1 - F_v) \mathbf{c}_d}{\pi} + D V F \quad (3.24)$$

$$\langle \mathbf{n} \cdot \mathbf{l} \rangle_{\text{Subsurface}} = \text{clamp} \left(\frac{\mathbf{n} \cdot \mathbf{l} + w}{1 + w}, 0, 1 \right), \quad w = 0.5 \quad (3.25)$$

The shininess exponent comes from roughness ρ ,

$$m = \max \left(\frac{2}{\rho^2 + 10^{-4}} - 2, 1 \right), \quad (3.26)$$

so roughness 0.5 gives $m = 6$. Phong's mirror direction $\mathbf{r}$ is cheap but needs a reflection per light; Blinn-Phong replaces it by $\mathbf{h}$, which is cheaper and behaves better at glancing angles, and the factors $(m + 2)/2\pi$ and $(m + 8)/8\pi$ normalise the highlight so it does not gain energy as it sharpens. *Oren-Nayar* treats a rough surface as many tiny Lambertian facets that scatter light back toward the source, so cloth and concrete look flat and do not darken at the edge like Lambert: with $\sigma = \rho$, $A = 1 - \frac{0.5\sigma^2}{\sigma^2 + 0.33}$, $B = \frac{0.45\sigma^2}{\sigma^2 + 0.09}$, $\alpha = \max(\theta_i, \theta_r)$, $\beta = \min(\theta_i, \theta_r)$ and $\Delta\varphi$ the azimuth between light and view around the normal. *Subsurface* lets light wrap a little past the shadow line (the cosine ramps from 1 down to 0 as $\mathbf{n} \cdot \mathbf{l}$ goes from 1 to $-w$), imitating light scattering inside skin. *Cook-Torrance* is the physically based default: D is the GGX facet distribution, V the Smith visibility and F Schlick's Fresnel,

$$D = \frac{\alpha^2}{\pi ((\mathbf{n} \cdot \mathbf{h})^2 (\alpha^2 - 1) + 1)^2}, \quad \alpha = \rho^2, \quad (3.27)$$

$$F = F_0 + (1 - F_0) (1 - \mathbf{v} \cdot \mathbf{h})^5,$$

$$V = \frac{0.5}{(\mathbf{n} \cdot \mathbf{l}) \sqrt{(\mathbf{n} \cdot \mathbf{v})^2 (1 - \alpha^2) + \alpha^2} + (\mathbf{n} \cdot \mathbf{v}) \sqrt{(\mathbf{n} \cdot \mathbf{l})^2 (1 - \alpha^2) + \alpha^2}}.$$

In the game, cloth and canvas use Oren–Nayar, skin uses Subsurface, and everything else uses Cook–Torrance (Table 3.1).

Table 3.1: The six illumination models compared

Model	Idea and best use	Cost
Lambert	Matte diffuse only; the baseline and the particle smoke	Lowest
Phong	Diffuse plus a mirror-direction highlight; plastic, teaching reference	Low
Blinn–Phong	Highlight from the half vector; stable at glancing angles	Low
Oren–Nayar	Rough matte facets; cloth, canvas, soldiers’ uniforms	Medium
Cook–Torrance	Microfacets with Fresnel; metal, paint, concrete, glass (default)	Higher
Subsurface	Wrapped cosine; skin and soft materials	Low

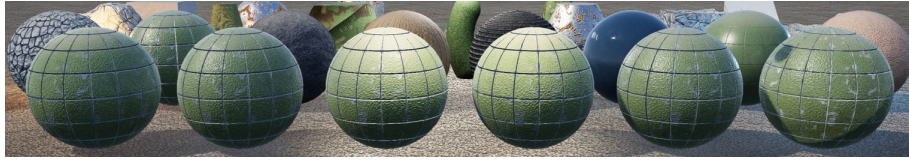


Figure 3.7: The same painted-metal sphere under the six illumination models (left to right: Lambert, Phong, Blinn–Phong, Oren–Nayar, Cook–Torrance, Subsurface), lit by the same sun and sky. Lambert has no highlight; Phong and Blinn–Phong add a hot spot; Oren–Nayar flattens the falloff; Cook–Torrance adds Fresnel and a microfacet highlight.

Light sources (Fig. 3.6b) differ only in how $L_j \text{ att}_j$ is found. A *directional* light (sun, moon) has no falloff and parallel rays. A *point* light (lamp, muzzle flash, explosion) uses the inverse-square law multiplied by a smooth window so it reaches exactly zero at its range R and can be drawn as a bounded sphere:

$$\text{att}(d) = \begin{cases} \frac{(1 - (d/R)^4)^2}{d^2 + 1}, & 0 \leq d \leq R, \\ 0, & d > R. \end{cases} \quad (3.28)$$

A *spot* light (floodlight, flashlight) multiplies this by a cone factor

$$c(\theta) = \begin{cases} 1, & \theta \leq \theta_i, \\ \left(\frac{\cos \theta - \cos \theta_o}{\cos \theta_i - \cos \theta_o} \right)^2, & \theta_i < \theta < \theta_o, \\ 0, & \theta \geq \theta_o, \end{cases} \quad (3.29)$$

where θ is the angle from the spot axis, θ_i the inner and θ_o the outer cone angle. An *area* light (ceiling panel) is a rectangle sampled as a 4×4 grid of point emitters, each with intensity/16 and weighted by its emission cosine, which gives soft highlights and soft shading. For $R = 14$ m: $\text{att}(1) = 0.50$, $\text{att}(5) = 0.037$, $\text{att}(10) = 0.0054$ and $\text{att}(14) = 0$ (checked by script, Fig. 3.8).

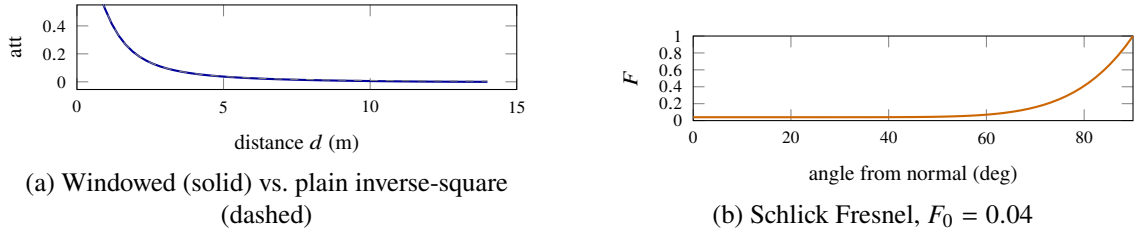


Figure 3.8: Two curves behind the lighting: distance falloff and glancing-angle reflection.

Shadows. The sun's shadow uses three *cascaded shadow maps*: the camera's view range is cut at

$$z_i = \lambda n \left(\frac{r}{n} \right)^{i/N} + (1 - \lambda) \left(n + (r - n) \frac{i}{N} \right), \quad i = 1, \dots, N, \quad (3.30)$$

a blend of logarithmic cuts (sharp shadows nearby) and even cuts, with $\lambda = 0.75$, $N = 3$, $n = 0.1$ m, $r = 100$ m, giving 9.1, 24.2 and 100 m. A pixel is looked up in the cascade containing its depth, moved a little along its normal (removes self-shadow acne), and compared against the stored depth with eight rotated percentage-closer taps, so the edge is soft. Spot lights have their own shadow maps, and *screen-space ambient occlusion* (SSAO) darkens corners where the sky is hidden.

Ambient light is the sky seen from the surface. Diffuse ambient is the sky's cosine-weighted colour; specular ambient uses Karis's split-sum fit, prefiltered sky $\times (F_0 A + B)$, where (A, B) depends only on roughness and $\mathbf{n} \cdot \mathbf{v}$; rough surfaces see a blurred sky. Both are multiplied by the occlusion term AO and fade inside rooms (the room volumes mask the sky), so interiors are lit by their lamps and the light entering by doors.

3.2.6 Sky, Time of Day and Clouds

The sun's height follows astronomy,

$$\sin(\text{elevation}) = \sin \varphi \sin \delta + \cos \varphi \cos \delta \cos h, \quad h = 15^\circ(\text{hours} - 12), \quad (3.31)$$

with latitude φ , declination δ and hour angle h . The sky colour comes from Rayleigh and Mie scattering [6]. Clouds live on two planes, cumulus at $H = 1500$ m and cirrus at 7 km. A view ray with direction $\mathbf{d}$ meets the plane at

$$\mathbf{P}_{xz} = \frac{H}{d_y} (d_x, d_z) + \mathbf{w} t, \quad (3.32)$$

where $\mathbf{w}$ is the wind velocity: shifting the noise by $\mathbf{w}t$ makes the clouds drift.

3.2.7 Terrain, Trees, Physics and Gameplay

Terrain is warped, ridged noise on a base plateau, streamed in 64 m chunks. **Trees** (oak, pine, birch, cypress) are tables of limbs and leaf lumps; each instance gets its own size, lean and tint, and the vertex shader bends it as a cantilever,

$$\Delta_x = 2s (0.04 + 0.1g) \left(\frac{y}{9}\right)^2, \quad \Delta_z = 0.35 \Delta_x, \quad g = \frac{1}{2} + \frac{1}{2} \sin(1.3t + 0.05x + 0.03z), \quad (3.33)$$

where s is the sway strength and g a travelling gust.

Collision uses rotatable boxes and a capsule player. Cars follow the bicycle model, and engine power fades toward top speed:

$$\dot{\psi} = \frac{v \tan \delta}{L}, \quad \dot{v} = a_0 \left(1 - 0.6 \frac{|v|}{v_{\max}}\right), \quad (3.34)$$

with yaw ψ , speed v , steering δ , wheel base L . **Ladders** are a four-state machine (mounting, climbing, topping out), owned by the controller so it cannot stick. Five mission goals and all sounds are code-driven.

CHAPTER IV

Implementation, Results and Discussions

4.1 Introduction

This chapter states how the work was measured and shows what it produces.

4.2 Experimental Setup

An Apple-silicon Mac (macOS, OpenGL 4.1 core), release build (`-o: speed`): 164 Odin files (about 19 200 lines) and 47 GLSL files (about 2 000 lines). Benchmark: a scripted 1 km flight at 1080p, 300 frames (Table 4.1).

Table 4.1: Experimental setup

Item	Value
Machine	Apple-silicon Mac, macOS, OpenGL 4.1 core profile
Build	Odin dev-2026-09, <code>-o: speed</code>
Code size	164 Odin files (about 19 200 lines), 47 GLSL files (about 2 000 lines)
Resolution	1920×1080 benchmark; 2560×1440 screenshots

4.3 Evaluation Metrics

Frame time $t = 1000/\text{fps}$ ms (budget 16.7 ms) and GPU time per pass; test pass or fail; and tiling error, which must be zero:

$$\varepsilon = \max_{\mathbf{u}} |f(\mathbf{u} + \mathbf{k}) - f(\mathbf{u})|, \quad \mathbf{k} \in \mathbb{Z}^2. \quad (4.1)$$

4.4 Dataset

No dataset is used. All content comes from fixed seeds, so every run is reproducible and no image, model or recording is stored.

4.5 Implementation and Results

4.5.1 Quantitative results

The display locks the average frame to 16.75 ms (60 Hz); the GPU work needs only 10.91 ms (Table 4.2, Fig. 4.1). Verification: 281 unit tests (noise, meshes, IK, collision, ladders, vehicles, mission, audio), 30 GL checks, 8 318 texture checks (exact tiling of every material), 318 567 renderer checks (lighting, shadows, atmosphere, instancing) and two scripted playthroughs that both finish 5 of 5 objectives.

Table 4.2: GPU time per pass, 1080p benchmark

Pass	Time (ms)	Share
Geometry (G-buffer)	4.11	38%
Lighting (sun, ambient, reflections, fog)	2.79	26%
Shadows (3 cascades)	1.58	14%
Post-processing	1.21	11%
Ambient occlusion	1.20	11%
Sky table	0.02	0%
GPU total	10.91	100%

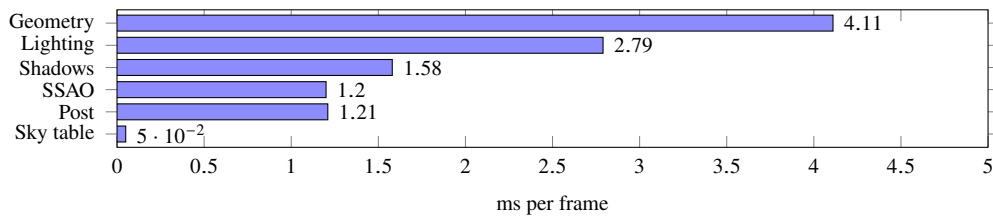


Figure 4.1: GPU time per pass (total 10.91 ms of the 16.7 ms budget).

4.5.2 Qualitative results

Figure 4.2 shows the running game: the base, a firefight with alarm, the automatic material showroom, IK-posed soldiers, tree species, drifting clouds and hands on a ladder. Panels (h) and (i) of Fig. 4.2 document a bug: the fog colour sampled the whole sky, so a horizon star painted a vertical line on every surface in its column (left of centre); the fog now uses the smooth atmosphere only.

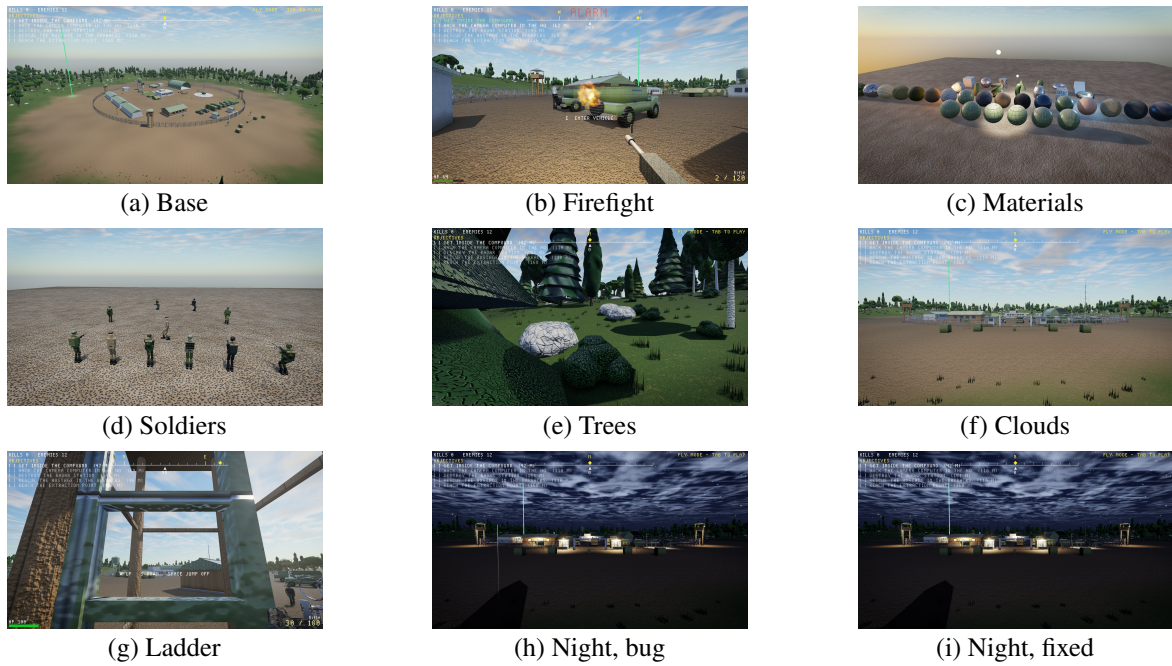


Figure 4.2: The engine and game in action; (h) and (i) show the streak bug and its fix.

4.5.3 Analysis of the results

Geometry costs most (38%), so instancing and per-chunk culling matter; the GPU uses two thirds of the budget. The streak was found by bisecting draw items, lights and effects.

4.6 Objective Achieved

(1) Six layers, downward-only. (2) No model, image or sound file. (3) Deferred PBR with shadows, SSAO, reflections, sky, bloom, ACES, FXAA. (4) Two playable missions. (5) 281 tests, 327 000 GL checks, 10.9 ms GPU time.

CHAPTER V

Societal, Health, Environment, Safety, Ethical, Legal and Cultural Issues

5.1 Intellectual Property Considerations

All code, shapes, textures and sounds in Sentinel are original: because every asset is generated from formulas and seeds, there are no third-party models, images or recordings to license. The libraries used (Odin, GLFW, miniaudio) are open source under permissive licences, and the papers and notes listed in the references are cited wherever their ideas are used. The mission is inspired by, but not copied from, *Project I.G.I.*: no names, maps or art are reused.

5.2 Ethical Considerations

The game is a work of fiction about an invented compound. Its figures are abstract, low-detail shapes, and it contains no real places, groups or people. It is intended for adult players. No personal data is collected, no network connection is made, and saved progress is a small local text file that the player can read or delete.

5.3 Safety Considerations

Muzzle flashes, explosions and the alarm can bother light-sensitive players, and loud sound can hurt hearing. For this reason brightness changes are smoothed by an eye-adaptation curve, flashes are short, and all sounds are mixed and limited before reaching the speakers. Players are advised to take regular breaks, and the pause menu is always available with one key. The game also never forces fast reaction: enemies give an audible warning before an alarm, and crouching makes the player quieter and harder to see.

5.4 Legal Considerations

Open-source licence terms are respected, and no proprietary code, trademark, logo or map is included. Violence is stylised, and the project makes no claim to real military accuracy. The report credits all cited work, and the course rules on originality are followed: the code was written for this course, and the earlier course version is kept as a clearly marked git tag.

5.5 Impact on Societal, Health and Cultural Issues

An open, readable engine lowers the barrier to learning graphics: a student can read one short file per feature, change a number and see its effect at once. This supports the course aim of understanding rendering by building it, and the table-driven content lets non-programmers adjust shapes and materials. Shooter games can encourage long play sessions, so the design favours stealth, crouching and rescue over constant shooting; the player is rewarded for avoiding detection and for freeing the hostage. No cultural or religious symbols are used, and the briefing text is neutral and fictional. The tactical map and compass help players who find navigation hard.

5.6 Impact on the Environment and Sustainability

Because no art files are downloaded, the whole project is a few megabytes of source code, which reduces storage and transfer energy; a conventional game of similar scope would ship hundreds of megabytes of models and textures. Display synchronisation caps the frame rate at the monitor's refresh rate and avoids needless GPU work, and the GPU finishes each frame in about two thirds of its time budget, so the laptop is not driven at full load.

CHAPTER VI

Addressing Complex Engineering Problems and Activities

6.1 Complex Engineering Problems

Table 6.1 maps the project to the usual attributes of a complex engineering problem. The requirements pull against each other: realism needs many passes, the budget allows 16.7 ms, “no files” removes easy detail, and the platform removes compute shaders.

Table 6.1: Complex engineering problem attributes

Attribute	In this project
Depth of knowledge	Radiometry, noise theory, numerical filters, GPU pipelines, collision, state machines, sound synthesis.
Conflicting requirements	Realism versus a 16.7 ms frame; no files versus rich surfaces; OpenGL 4.1 without compute shaders.
Depth of analysis	Bisecting the fog star streak; proving that textures tile exactly; tuning physics curves.
Interdependence	Renderer, world, game and audio share data layouts and timing.
Novelty	A complete game from code and seeds only, written to be read.

6.2 Complex Engineering Activities

The project used modern resources (compiler, GPU timers, test runners, version control) and many interacting components: a fog change affected the sky, clouds, stars and night lights. Quality was verified by measurement rather than by eye alone (Fig. 6.1), with unit tests, GL checks and scripted playthroughs run before each commit.

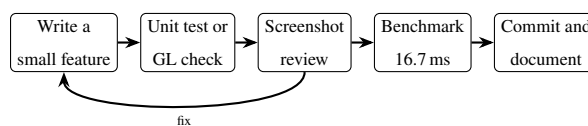


Figure 6.1: The verification loop used for every feature.

CHAPTER VII

Conclusions

7.1 Summary

Sentinel shows that a complete 3D game, from surfaces and shapes to weather and sound, can be generated entirely by code under an OpenGL 4.1 limit: a six-layer engine, a deferred physically based renderer, noise-based recipes, table-driven content, a mission game and a large test suite fit in about 21 000 lines and run within a 60 fps budget (10.9 ms of GPU time). Most of what looks like “art” is a few simple ideas applied consistently and tested.

7.2 Limitations

Models are blocky (primitives plus noise), and characters are simple. There is no true global illumination or ray tracing; indirect light and reflections are screen-space approximations. Foliage is lumpy volume, the beacon is a solid column, soldiers cannot pick up weapons, and tuning used scripts and screenshots.

7.3 Recommendations and Future Works

The engine can be extended with leaf cards with cut-out transparency for more realistic trees, screen-space global illumination, point-light shadows, anisotropic highlights for brushed metal, networked multiplayer and a larger user study of difficulty and comfort. A port to a modern graphics API with compute shaders would allow noise, terrain and even animation to be generated on the GPU, and would allow true ray-traced reflections.

REFERENCES

- [1] K. Perlin, “Improving noise,” *ACM Trans. Graph. (SIGGRAPH)*, vol. 21, no. 3, pp. 681–682, 2002.
- [2] S. Worley, “A cellular texture basis function,” in *Proc. SIGGRAPH*, 1996, pp. 291–294.
- [3] R. L. Cook and K. E. Torrance, “A reflectance model for computer graphics,” *ACM Trans. Graph.*, vol. 1, no. 1, pp. 7–24, 1982.
- [4] B. Walter, S. R. Marschner, H. Li and K. E. Torrance, “Microfacet models for refraction through rough surfaces,” in *Proc. Eurographics Symp. Rendering*, 2007, pp. 195–206.
- [5] B. Karis, “Real shading in Unreal Engine 4,” SIGGRAPH Course: Physically Based Shading in Theory and Practice, 2013.
- [6] S. Hillaire, “A scalable and production ready sky and atmosphere rendering technique,” *Computer Graphics Forum*, vol. 39, no. 4, pp. 13–22, 2020.
- [7] R. Dimitrov, “Cascaded shadow maps,” NVIDIA White Paper, 2007.
- [8] M. Oren and S. K. Nayar, “Generalization of Lambert’s reflectance model,” in *Proc. SIGGRAPH*, 1994, pp. 239–246.
- [9] K. Narkowicz, “ACES filmic tone mapping curve,” blog post, 2016.
- [10] J. Jimenez, “Next generation post processing in Call of Duty: Advanced Warfare,” SIGGRAPH Course: Advances in Real-Time Rendering, 2014.
- [11] T. Lottes, “FXAA,” NVIDIA White Paper, 2009.
- [12] M. Segal and K. Akeley, *The OpenGL Graphics System: A Specification, Version 4.1*, Khronos Group, 2010.
- [13] I. Quilez, “Domain warping,” online article, iquilezles.org.
- [14] S. Hargreaves and M. Harris, “Deferred shading,” NVIDIA presentation, Game Developers Conference, 2004.
- [15] H. Scharr, “Optimal operators in digital image processing,” Ph.D. dissertation, Univ. Heidelberg, 2000.
- [16] T. Akenine-Möller, E. Haines and N. Hoffman, *Real-Time Rendering*, 4th ed., CRC Press, 2018.